PHISHGUARD: Web based phishing Detection System

An Engineering Project in Community Service

Phase – II Report

Submitted by

Hamza Ali Ashrafi (23BCY10069)

in partial fulfillment of the requirements for the degree of

Bachelor of Technology



VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradhesh

March, 2026

Declaration of Originality

We, hereby declare that this report entitled **Gen AI for Security** represents our original work carried out for the EPICS project as a student of VIT Bhopal University and, to the best of our knowledge, it contains no material previously published or written by another person, nor any material presented for the award of any other degree or diploma of VIT Bhopal University or any other institution. Works of other authors cited in this report have been duly acknowledged under the section "References".

Date : March 30, 2026

Reg No & Name

Hamza Ali Ashrafi

23BCY10069

Acknowledgement

This work has benefited in various ways from several people. We would like to express our heartfelt gratitude to all those who have supported us throughout the development of the PhishGuard project.

We extend our sincere thanks to our project supervisor Dr. Ashok Patel for their invaluable guidance, constant encouragement, and constructive feedback throughout the course of this project. Their expertise in the domain of cybersecurity and machine learning was instrumental in shaping the direction and depth of this work.

We are grateful to the Department of Computer Science and Engineering, VIT Bhopal University, for providing us with the necessary infrastructure, resources, and academic environment to pursue this project. The faculty members and lab technicians have been extremely cooperative and helpful.

We acknowledge the open-source community and dataset providers – Kaggle, the Enron Email Dataset contributors, and the Phishing Email Detection Dataset maintainers – whose publicly available datasets formed the empirical backbone of our machine learning model.

Finally, we thank our family and friends for their unwavering moral support, patience, and encouragement throughout this journey.

Abstract

PhishGuard is a real-time phishing detection system developed as a Chrome extension to safeguard users from malicious messages across web platforms such as Gmail and WhatsApp Web. The project integrates Machine Learning techniques with rule-based analysis to identify phishing attempts effectively. The system extracts message content dynamically, processes it through a trained classification model via an API, and generates a hybrid phishing score to determine whether a message is safe or malicious.

The overall system architecture consists of multiple modules, including dataset preprocessing, feature engineering, model training, backend API development, Chrome extension integration, hybrid detection logic, and user interface design. The Machine Learning model, trained using the XGBoost algorithm, analyzes textual features and custom phishing indicators to classify messages. The FastAPI-based backend serves predictions, while the Chrome extension enables real-time interaction with user data.

My primary contribution to this project was focused on the **Chrome Extension Module (Core Interaction)** and **integration of the extension with the backend API**. This involved building the communication pipeline between the browser environment and the machine learning model, ensuring seamless real-time detection. I worked on extracting message content from web pages, handling asynchronous communication between scripts, managing API requests via the background script, and displaying detection results to users.

- This report highlights my individual contribution in detail while also providing an overview of the complete system.

❖ Introduction

Phishing attacks are one of the most common cybersecurity threats, targeting users through deceptive messages that attempt to steal sensitive information such as passwords, banking details, or personal data. With the increasing use of web-based communication platforms, users are more vulnerable to such attacks.

PhishGuard aims to address this issue by providing a **lightweight, browser-based real-time phishing detection system**. Unlike traditional antivirus solutions, PhishGuard operates directly within the browser, analyzing messages as users interact with them.

The system leverages:

- Machine Learning for intelligent classification
- Rule-based analysis for detecting known phishing patterns
- Chrome Extension APIs for real-time interaction

❖ Overview of the Complete System

The PhishGuard system is divided into several modules:

➢ Dataset and Preprocessing

- SMS Spam Detection Dataset was used
- Data labeled as:
  - *ham* (legitimate)
  - *spam* (phishing)
- Preprocessing included:
  - Lowercasing
  - Removal of special characters
  - URL extraction
  - Tokenization

➢ Feature Engineering

Two major feature types:

- **Text Features**: TF-IDF Vectorization
- **Custom Features**:
  - URL count
  - Suspicious keywords
  - Message length
  - Shortened link detection

- **Model Training**
  - Algorithm: XGBoost Classifier
  - Evaluation metrics:
    - Accuracy
    - Confusion Matrix
    - Classification Report

- **Backend API**
  - Framework: FastAPI
  - Endpoint: /predict
  - Returns:
    - Prediction (phishing/safe)
    - Probability score

- **Hybrid Detection Logic**

Final decision based on:

  - Machine Learning score (60%)
  - Rule-based score (40%)

- **User Interface**
  - Chrome popup displays logs
  - Stores detected threats
  - Allows clearing of logs

My Contribution

Chrome Extension Module (Core Interaction)
My primary responsibility was to design and implement the **core interaction layer of the Chrome extension**, which acts as a bridge between the user interface and the backend intelligence system.

1. Chrome Extension Architecture

The Chrome extension consists of the following key components:

1.1 content.js

- Injected into web pages (Gmail, WhatsApp Web)
- Detects and extracts message content from the DOM
- Sends extracted data to the background script

1.2 background.js

- Acts as an intermediary between content script and API
- Handles API requests
- Avoids browser security issues (like mixed content errors)

1.3 manifest.json

- Defines permissions
- Registers scripts
- Configures extension behavior

2. Message Detection and Extraction

One of my key tasks was implementing **real-time message detection** from web pages.

Challenges Faced

- Gmail's dynamic DOM structure
- Frequent UI updates
- Capturing only relevant message content

Solution Implemented

- Used DOM observers (MutationObserver)
- Applied refined selectors to filter actual messages
- Avoided capturing irrelevant UI elements

This ensured accurate extraction of:

- Email body text
- Chat messages
- Newly incoming messages

3. Communication Between Scripts

Chrome extensions follow a multi-layered architecture where scripts cannot directly communicate freely.

Implemented Workflow

1. content.js detects message
2. Sends message to background.js using chrome.runtime.sendMessage()
3. background.js processes the request

This design ensures:

- Better separation of concerns
- Secure communication
- Scalable architecture

4. API Integration (Core Responsibility)

A major part of my contribution was connecting the Chrome extension to the backend API.

Key Responsibilities

- Sending extracted text to API
- Handling asynchronous requests
- Processing API responses

Implementation Details

- Used fetch() inside background.js
- Sent POST request to:

http://127.0.0.1:8000/predict

- JSON payload:

{ "text": "message content" }

Response Handling

- Extracted:
  - Prediction
  - Probability score
- Forwarded response back to content.js

5. Handling Browser Security Constraints

Problem

- Direct API calls from content.js caused:
  - Mixed content errors (HTTP vs HTTPS)

Solution

- Implemented **background script as API proxy**
- All API calls routed through background.js

This ensured:

- Secure communication
- Compliance with browser policies

6. Real-Time Detection Workflow

The complete workflow implemented by me:

1. User opens Gmail/WhatsApp Web
2. content.js detects new message
3. Extracts message text
4. Sends to background.js
5. background.js calls API
6. Receives prediction
7. Sends result back
8. content.js displays alert

7. Integration with Hybrid Detection Logic

Although the hybrid scoring logic was part of the overall system, **my role included:**

- Passing API results correctly
- Enabling rule-based checks in extension
- Supporting final decision-making process

This ensured:

- Improved accuracy
- Reduced false positives

8. Challenges Faced and Solutions

1. Incorrect Message Capture
- Issue: Capturing UI elements instead of messages
- Solution: Improved DOM selectors

2. API Communication Errors
- Issue: Failed requests
- Solution: Structured request handling

3. Mixed Content Blocking
- Issue: HTTPS vs HTTP conflict
- Solution: Background script proxy

4. Real-Time Performance
- Issue: Delays in detection
- Solution: Optimized message triggering

9. Outcome of My Contribution

Through my work:
- Enabled real-time phishing detection
- Connected frontend with backend intelligence
- Ensured smooth and secure communication
- Improved system reliability

My module served as the core operational layer of the entire project, without which the system would not function in real-time.

REFERENCES

1. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794

2. Anti-Phishing Working Group (APWG). (2023). Phishing Activity Trends Report, Q4 2023. Available: https://apwg.org/trendsreports/

3. Choudhary, N., & Bhatt, C. (2021). Phishing websites detection using machine learning. Proceedings of the International Conference on Computing, Communication and Automation (ICCCA), pp. 301–305.

4. Google. (2024). Chrome Extensions Manifest V3 Overview. Availble:https://developer.chrome.com/docs/extensions/mv3/intro/

5. Kaggle. (2023). SMS Spam Collection Dataset. Available: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset 30 17.

6. Kaggle. (2023). Enron Email Dataset. Available: https://www.kaggle.com/datasets/wcukierski/enron-email-dataset 18.

7. Kaggle. (2023). Phishing Email Detection Dataset. Available: https://www.kaggle.com/datasets/subhajournal/phishingemails

8. alloum, S., Gaber, T., Vadera, S., & Shaalan, K. (2021). A systematic literature review on phishing email detection using natural language processing techniques. IEEE Access, 10, 65703–65727.

9. Zieni, R., Massari, L., & Calzarossa, M. C. (2023). Phishing or not phishing? A survey on the detection of phishing websites. IEEE Access, 11, 58891–58910.

Conclusion

The PhishGuard project demonstrates a practical application of cybersecurity concepts using Machine Learning and browser-based technologies. My contribution focused on building the bridge between user interaction and intelligent detection systems through the Chrome extension.

By successfully implementing real-time message extraction and API integration, I ensured that phishing detection happens seamlessly as users interact with web platforms. This module plays a critical role in delivering instant protection and enhancing user awareness.

The project not only strengthens technical skills in areas like JavaScript, Chrome APIs, and API integration but also contributes to solving a real-world cybersecurity problem.